

Safe low-level programming with Rust

CptS 355 — Fall 2026

Thomas Gilray

C/C++

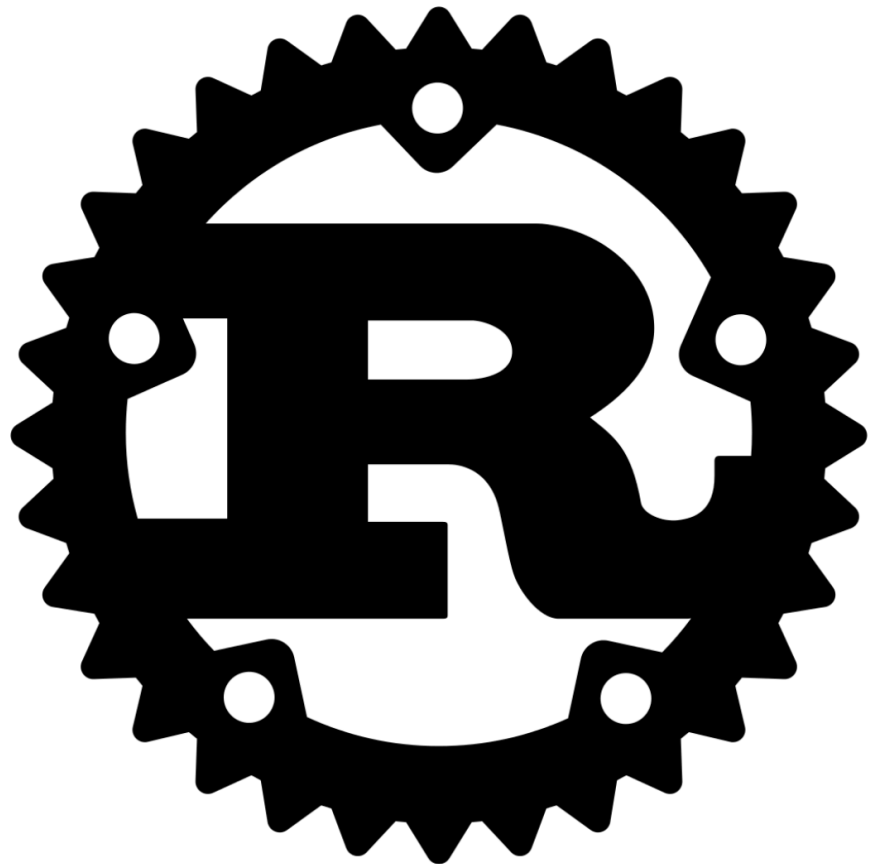
- Type unsafe
- Memory unsafe
- Manual / deterministic memory management
- Low-level control
- Performance over ease
- *Less* dynamic abstraction overhead

Python / Java / etc

- Type safe
- Memory safe
- Garbage collected
- High-level, expressive but w/ less control
- Ease of use over speed
- More dynamic abstraction overhead

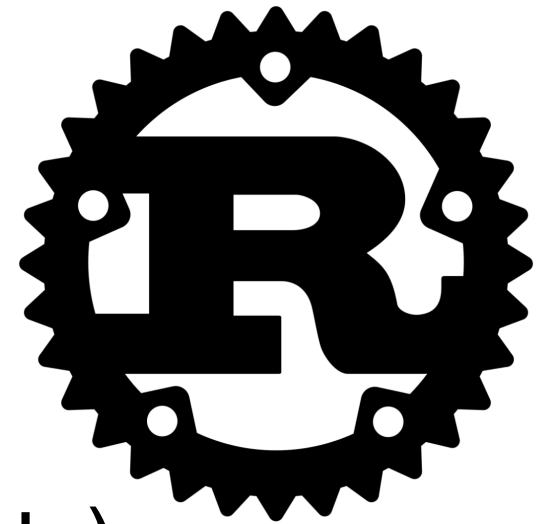
Wouldn't it be nice to have a safer low-level language for system programming?

Rust



- Type safe
 - Memory safe
 - Thread safe (no races)
 - Low-level control
 - Language-assisted manual / deterministic mem. management
 - Zero-cost abstractions
-
- Started at Mozilla by Graydon Hoare in 2006.
 - Intended as a successor to C++ for safe, low-level, performant system programming (e.g., browsers).

Rust Features



- Draws inspiration from OOP and FP
- **Ownership** and static **lifetimes** (memory safety)
- Separates **traits** as interfaces atop **implementations**
- Variables are **immutable** by default
- **Pattern matching** for data types
- Polymorphism:
 - **Ad hoc polymorphism** / static dispatch
 - **Subtype polymorphism** explicit via trait objects
 - Generics and **parametric polymorphism**
- **First-class functions**, closed automatically
- C bindings/FFI and unsafe operations where needed

More Resources

- Install: <https://www.rust-lang.org/en-US/install.html>
- The Rust Programming Language:
<https://doc.rust-lang.org/book/>
- Rust by example:
<https://doc.rust-lang.org/rust-by-example/>
- Too many lists:
<https://rust-unofficial.github.io/too-many-lists/>

Rust Basics: functions

```
// comment  
fn main()  
{  
    println!("Hello, world!");  
}
```

Hello, world!

Rust Basics: functions, conditionals

```
fn my_abs(value: i32) -> u32
{
    if value < 0
    {
        (-value) as u32
    }
    else
    {
        value as u32
    }
}
```

*Conditionals are expressions, unlike in C/C++,
and their return types must match!*

Rust Basics: let statements

```
{  
    let x = 10;  
    x * 10  
}  
// 100
```

```
{  
    let x = 13;  
    let x = 7 + x;  
    x  
}  
// 100
```

```
{  
    let x = 10;  
    x = x * 5;  
    x  
}  
// error
```

```
{  
    let mut x = 10;  
    x = x * 5;  
    x  
}  
// 50
```

```
{  
    let x:u32 = -1;  
    let y = x + 3;  
    y  
}  
// error
```

```
{  
    let x:i16 = -1;  
    let y = x + 3;  
    y  
}  
// 2
```

Rust Basics: factorial example

```
fn fact(n:i32) -> i32
{
    if n == 0 { 1 }
    else
    {
        let x = fact(n-1);
        n * x
    }
}
```

```
fn main() {
    let res = fact(6);
    println!("fact(6) = {}", res);
}
// fact(6) = 720
```

Rust Basics: quiz

```
{  
    let x : i64 = 6;  
    let y = "hi";  
    if x == 5 { y } else { 5 };  
    7  
}
```

What happens when we run this code?

5

6

7

Error

Rust Basics: quiz

```
{  
    let x : i64 = 6;  
    let y = "hi";  
    if x == 5 { y } else { 5 };  
    7  
}
```

What happens when we run this code?

5

6

7

Error

Rust Basics: quiz

```
{  
    let x = 99;  
    let y = 44;  
    y = x;  
    x == y  
}
```

What happens when we run this code?

99

true

false

Error

Rust Basics: quiz

```
{  
    let x = 99;  
    let y = 44;  
    y = x;  
    x == y  
}
```

What happens when we run this code?

99

true

false

Error

Rust Basics: loops

```
fn fact(n: u32) -> u32
{
    let mut x = n;
    let mut a = 1;
    loop
    {
        if x <= 1 { break; }
        a = a * x;
        x = x - 1;
    }
    a
}
```

Rust Basics: loops

```
fn fact(n: u32) -> u32
{
    let mut a = 1;

    for x in 1..n+1 // or 1..=n
    {
        a = a * x;
    }

    a
}
```

Rust Basics: quiz

```
{  
    let mut x = 1;  
    for i in 1..5  
    {  
        let x = x * 2;  
    }  
    x  
}
```

What happens when we run this code?

1

16

32

Error

Rust Basics: quiz

```
{  
    let mut x = 1;  
    for i in 1..5  
    {  
        let x = x * 2;  
    }  
    x  
}
```

What happens when we run this code?

1

16

32

Error

Rust Basics: more types

- `i8`, `i16`, `i32`, `i64`, `isize`, `u8`, `u16`, `u32`, `u64`, `usize`
- `char` (32bits to support unicode, without utf-8)
- `bool` = {`true`, `false`}
- `f32`, `f64` (floating point numbers)
- Note that arithmetic is overloaded for these types
- Fixed-width tuple types (`t0`, `t1`, `t2`, ...)
 - The empty tuple type `()` is `unit` (returns nothing)
- Accessed via the dot operator like a field { `tup.0` }
- Arrays of type `[t_elem; length]`: e.g., `[i32; 2]`

Rust Basics: pattern matching

```
fn dist(p0:(f64,f64), p1:(f64,f64)) -> f64
{
    let (x0,y0) = p0;
    let x1 = p1.0;
    let y1 = p1.1;
    let dx = x1 - x0;
    let dy = y1 - y0;
    (dx*dx + dy*dy).sqrt()
}
```

Rust Basics: pattern matching

```
fn dist((x0, y0): (f64, f64),  
        (x1, y1): (f64, f64)) -> f64  
{  
    let dx = x1 - x0;  
    let dy = y1 - y0;  
    (dx*dx + dy*dy).sqrt()  
}
```

Exercise: Collatz steps

```
fn collatz_steps(n: u32) -> u32
{
    // TODO ...
}
```

Rust Safety: memory safety

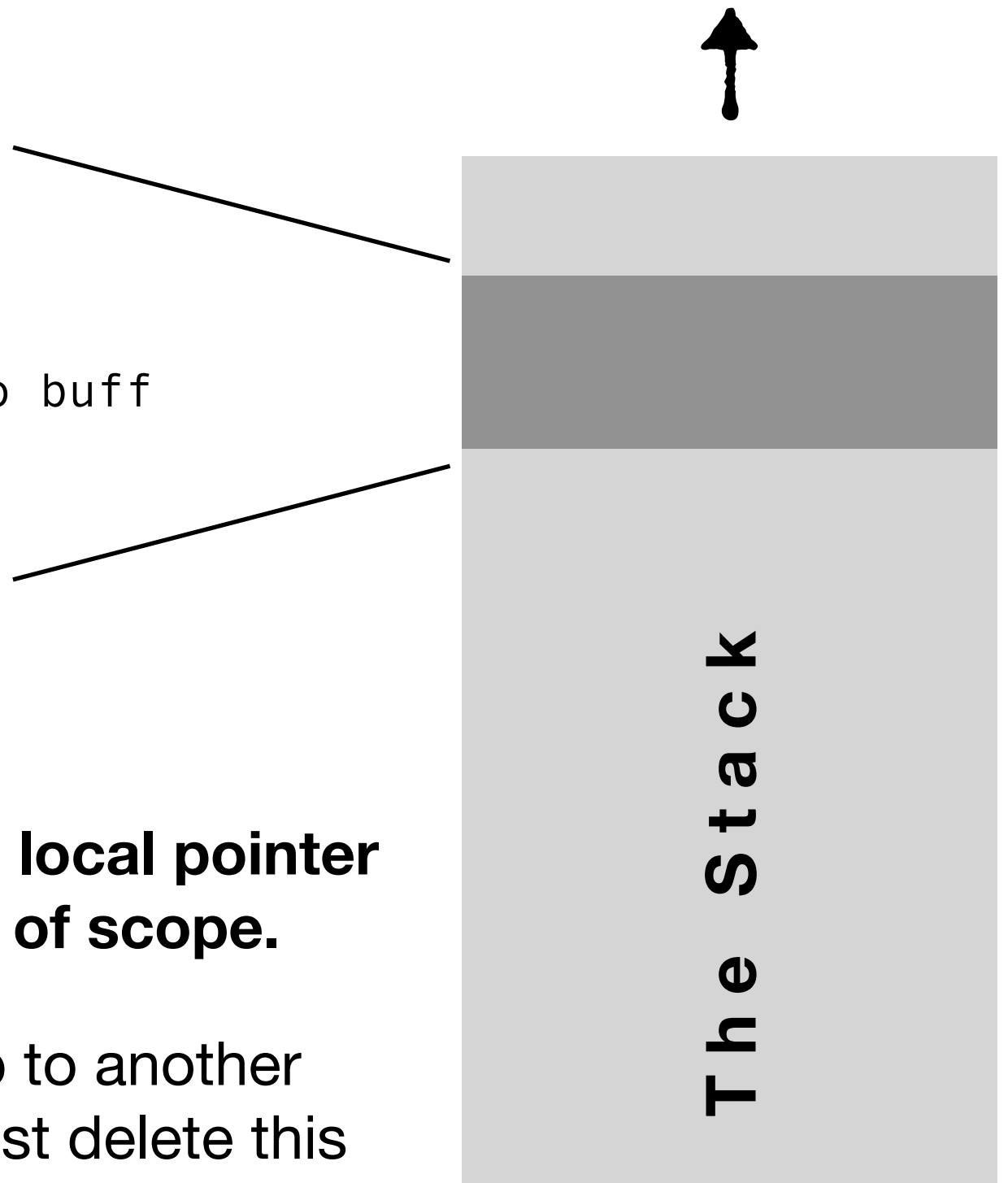
- Rust's heap is managed statically, deterministically and automatically, requiring neither delete/free nor GC.
- Type safety (no implicit type-related failures)
- Memory safety (no dangling pointers or memory leaks)
- Thread safety (no observable race conditions)
- Based around the idea that every value has a statically known lifetime, tied to a stack variable that owns it. When that variable goes out of scope, it is deleted.

Rust Safety: ownership and lifetimes

```
char* buff = new char[N];  
  
// ... some other function calls ...  
// may take a pointer or reference to buff  
  
delete [] buff;
```

In C/C++ we must explicitly delete a local pointer we've allocated once it goes out of scope.

Otherwise we must pass ownership to another variable and track who eventually must delete this value without any help from the language!

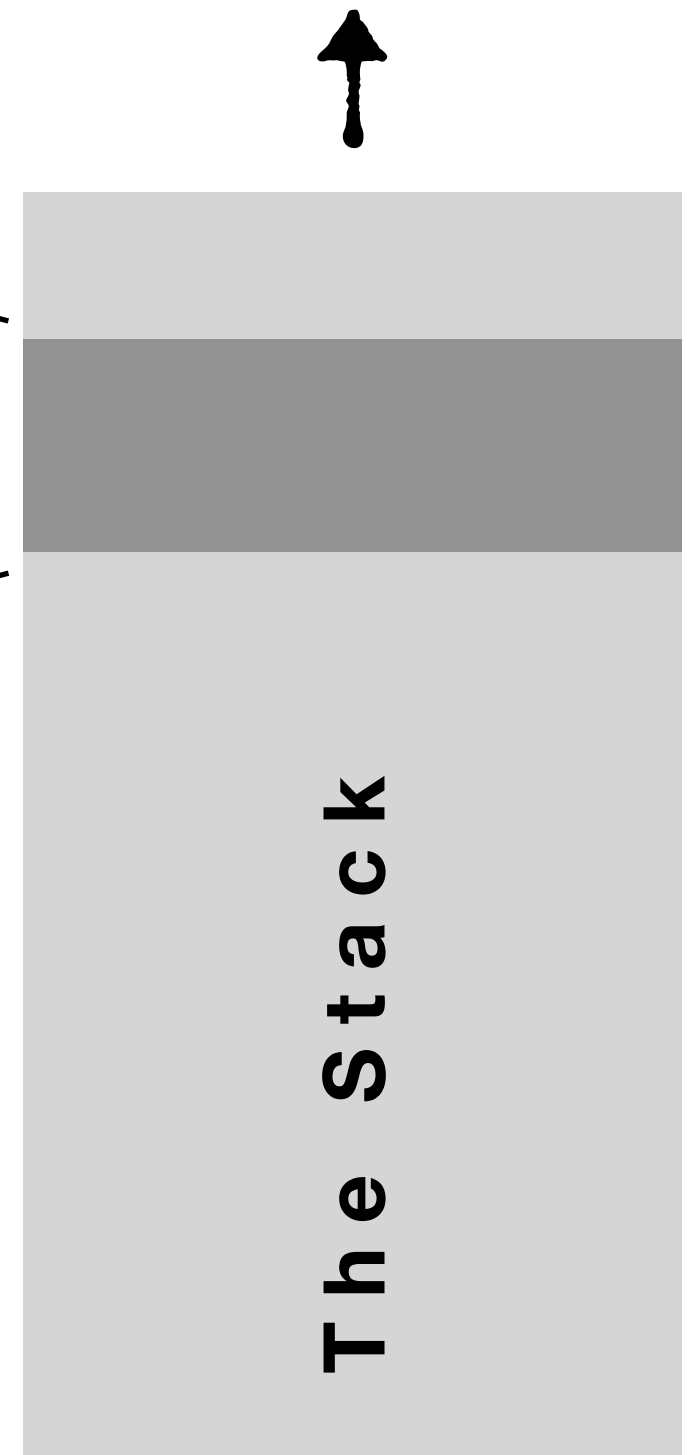


Rust Safety: ownership and lifetimes

```
String s = new String("Java");  
  
// ... some other function calls ...  
// may take a reference to s and safely keep it  
  
s = null;
```

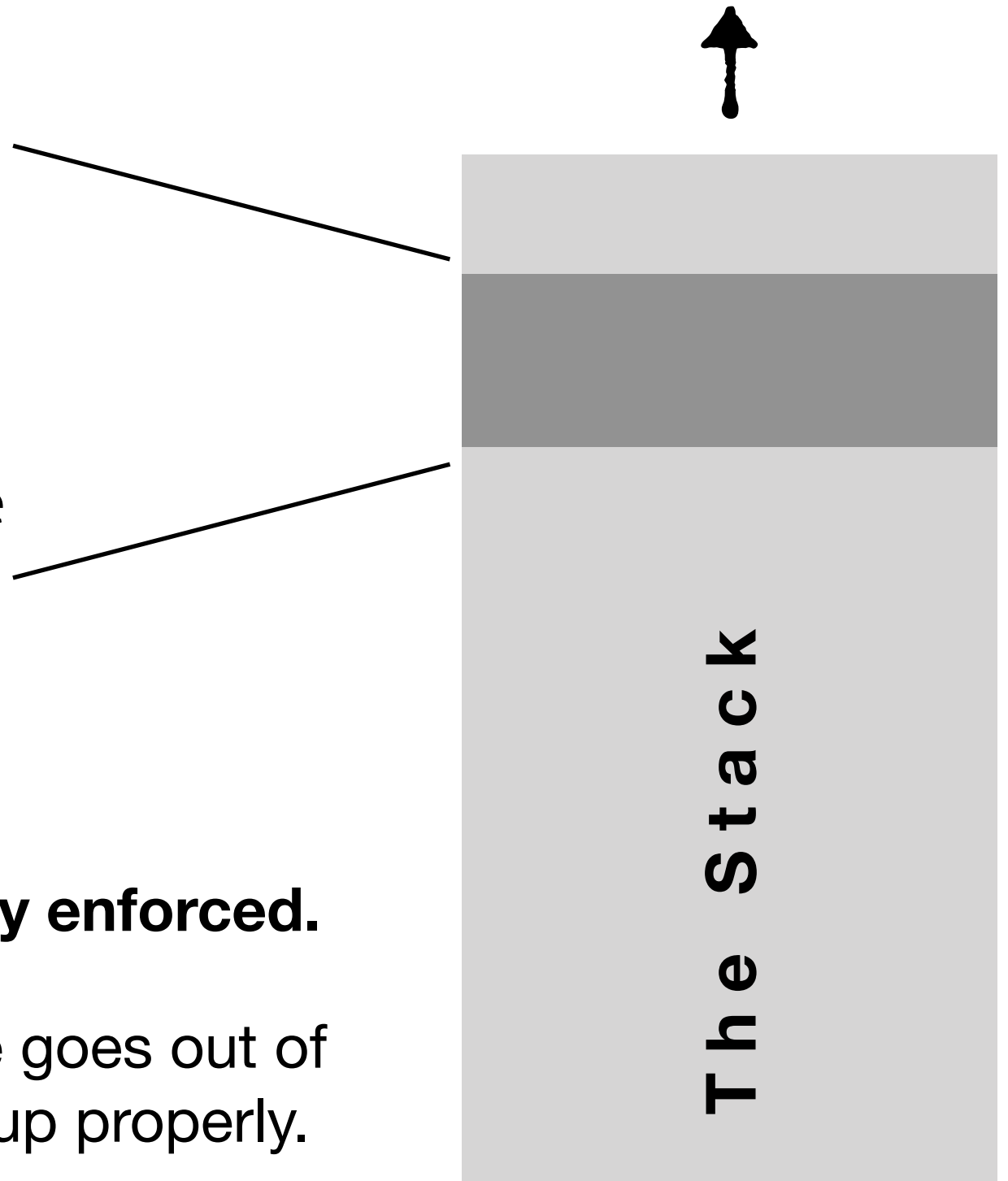
In Java/Python, memory safety is dynamically enforced at runtime via the Garbage Collector.

We may safely stash extra references to an object, to use later, and the GC will delete the object only when all references are unreachable.



Rust Safety: ownership and lifetimes

```
let s = String::from("Rust");  
  
// ... some other function calls ...  
// may borrow references to s safely  
  
// no cleanup at end of scope
```



In Rust, memory safety is statically enforced.

When the variable that ***owns*** a value goes out of scope, the value is always cleaned up properly.

Rust Safety: ownership and lifetimes

- In Rust, every value has exactly one variable ***owner***.
- When owner goes out of scope, the value is dropped.
- A value can be ***moved*** to a new owner or ***borrowed***.
- In this example, a local string is freed when we return from this function or nested scope:

```
{  
    let mut s = String::from("hello");  
    // s is the sole owner  
    s.push_str(", world!"); // Borrows s  
    println!("{}", s); // Borrows s  
}  
// s's data is freed by calling s.drop()
```

Rust Safety: ownership and lifetimes

- By default, an assignment moves (ownership) the data

```
let x = String::from("hello");  
let y = x;    // transfers ownership!
```

- As there can only be a single owner, x is now invalid

```
println!("{}", world!", y);    // OK  
println!("{}", world!", x);    // fails
```

- Otherwise, x and y would have to share ownership.
 - What issues could this cause if permitted?
- Primitives like u32, f64 (and anything deriving the Copy trait) are *copied* on assignment.

Rust: traits

- A trait is a way of saying what a type must support.
 - The Copy trait is a built-in that tells the compiler to copy a value instead of transferring ownership.
- Mostly traits are equivalent to interfaces in Java and specify a set of methods that must be defined.
 - The Clone trait requires a clone method.
 - The Deref trait requires a deref() method (for *x).
 - The Drop trait requires a user drop() method.
 - The PartialEq trait requires an eq() method.

Rust: traits

- Traits may be automatically derived in some cases using the `#[derive(T)]` annotation for a trait T. E.g.,
 - `#[derive(PartialEq)]` derives structural equality.
 - `#[derive(Debug)]` derives `dbg` conversion to string:

```
#[derive(Debug)] // Allows formatting with {:?}
struct Player {
    username: String, score: u32,
}

fn main() {
    let player = Player {
        username: String::from("bob"), score: 100 };
    println!("{:?}", player);
}
// Prints: Player { username: "bob", score: 100 }
```

Rust Safety: borrowing

- By default, a function call ***moves*** arguments to callee

```
fn id(s: String) -> String {  
    s  
}
```

- Ownership then changes hands in the caller:

```
fn main() {  
    let s1 = String::from("hello");  
    let s2 = id(s1); // s1 moved to arg  
    println!("{}", s2); // id's result moved to s2  
    println!("{}", s1); // fails  
}
```

Rust Safety: borrowing

- By default, a function call ***moves*** arguments to callee

```
fn stringlen(s: &String) -> usize {  
    s.len() // s (ref only) is dropped  
}
```

- Ownership then changes hands in the caller:

```
fn main() {  
    let s1 = String::from("hello");  
    let len = id(&s1); // s1 moved to arg  
    println!("{}", len); // OK  
    println!("{}", s1); // OK  
}
```

Rust Safety: borrowing

- In Rust we can borrow an immutable reference using `&x` and a mutable reference using `&mut x`
- The key principle is that you can only have one mutable reference borrowed at a time, OR any number of immutable references borrowed at a time.
- What is prevented by having this restriction?